

附件 3

西安电子科技大学

智能系统专业实验 课程实验报告

实验名称 实验 3-动物分类

XXXXXXXX 学院 2320xxx 班

姓名 XXXXXX 学号 XXXXXXXXXXXX

同作者 XXXXXXXX

实验日期 202x 年 x 月 x 日

成 绩

实验报告内容基本要求及参考格式

- 一、实验目的
- 二、实验环境
- 三、实验基本原理及步骤（模型描述、理论描述或方案描述）
- 四、实验自主创新（或改进）部分的描述
- 五、实验结果与分析（需给出结果图或表，并进行分析）
- 六、实验的收获及心得体会（或遇到的问题，如何解决）
- 七、实验代码

一、实验目的

1. 掌握图像分类任务的完整流程：数据集整理、数据增强、自定义 *Dataset*、卷积神经网络设计与训练。
2. 学会使用 *PyTorch* 加载自定义图像数据集，并按比例划分训练集与验证集。
3. 理解卷积神经网络的基本结构（卷积层、池化层、全连接层）及其在图像分类中的作用。

二、实验环境

1. 硬件: *SUB KIT NX* 嵌入式开发板
2. 操作系统: *Linux*
3. 软件及库:
 - 1) *Python*
 - 2) *NumPy*
 - 3) *PyTorch*
 - 4) *Matplotlib*
 - 5) *Torchvision*
 - 6) *Pillow (PIL)*
 - 7) *tqdm*

三、实验基本原理及步骤（模型描述、理论描述或方案描述）

1. 基本原理

本实验是一个二分类图像识别任务，目标是从图像中区分牛和羊。

1) 网络结构: 自定义的卷积神经网络 (*CNN*)

输入: $3 \times 128 \times 128$ 的 *RGB* 图像

第 1 个卷积层:

$\text{Conv2d}(3, 16, \text{kernel_size} = 3, \text{padding} = 1) \rightarrow \text{ReLU} \rightarrow \text{MaxPool2d}(2)$, 输出尺寸 $16 \times 64 \times 64$

第 2 个卷积层:

$\text{Conv2d}(16, 32, \text{kernel_size} = 3, \text{padding} = 1) \rightarrow \text{ReLU} \rightarrow \text{MaxPool2d}(2)$, 输出尺寸 $32 \times 32 \times 32$

展平后进入全连接层:

$\text{Linear}(32 \times 32 \times 32, 128) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.5) \rightarrow \text{Linear}(128, 2)$

2) 损失函数: 交叉熵损失。

- 3) 优化算法: *Adam*, 学习率 0.001。

2. 实验步骤

1) 数据集准备

Sample 下准备 *cow* 和 *sheep* 样本各 100 张图片, 共 200 张)。

2) 读取与划分数据

- a) 使用 *read_split_data()* 函数遍历文件夹, 获取所有图片路径。
- b) 生成类别到索引的映射 (*cow* $\rightarrow$ 0, *sheep* $\rightarrow$ 1), 并保存为 *.json* 文件。
- c) 按比例随机划分训练集和验证集, 返回路径列表和标签列表。

3) 自定义 *Dataset* 类

- a) *MyDataSet* 继承自 *torch.utils.data.Dataset*。
- b) *__getitem__* 中读取图片并转换 RGB, 应用预处理 (*Resize*, *ToTensor*)。
- c) 实现 *collate_fn* 用于批处理时堆叠张量。

4) 数据预处理与加载

- a) 将所有图片缩放到 128×128 像素。
- b) 使用 *DataLoader* 加载训练集和验证集, *batch_size* = 16。

5) 构建网络

- a) 定义 *Net* 类, 包含两个卷积块和两个全连接层
- b) 将模型移动到 *GPU*。

6) 训练网络

- a) 训练 10 个 *epoch*
- b) 每个 *epoch* 中, 对训练集迭代计算损失、反向传播、更新参数, 并记录平均损失和准确率。
- c) 每个 *epoch* 结束后在验证集上评估准确率。

7) 动态可视化

- a) 绘制训练损失、训练准确率、验证准确率随 *epoch* 变化的曲线
- b) 随机选取验证集中的若干样本, 显示真实标签与预测标签
- c) 保存模型权重为 *animal_classifier.pth*

四、实验自主创新 (或改进) 部分的描述

1. 增加 Dropout 层防止过拟合

在全连接层之间添加了 $Dropout(0.5)$ ，有效减少过拟合风险，提高模型泛化能力。

2. 详细的预测可视化

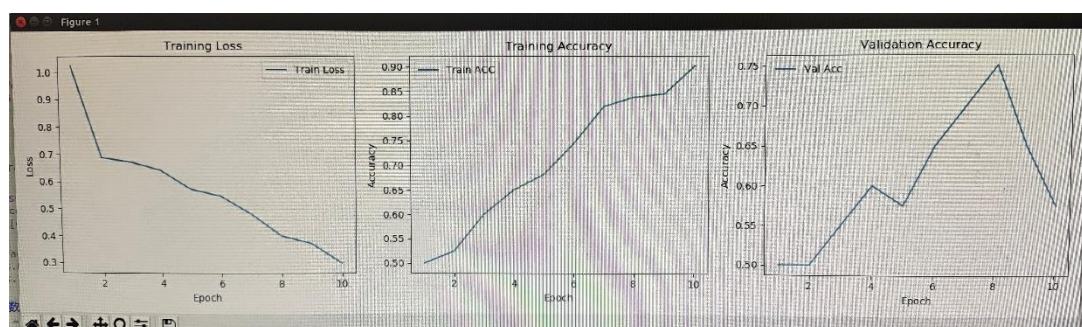
实现了 `visualize_predictions` 函数，能够展示验证集中多个样本的图像、真实标签和预测标签，便于直观评估模型效果。

3. 使用 *Adam* 优化器替代 *SGD*

相比实验指导书中的 *SGD*，*Adam* 具有自适应学习率，收敛更快，适合小规模图像分类任务。

五、实验结果与分析（需给出结果图或表，并进行分析）

1. 训练过程可视化



- 1) **左图 (*Training Loss*):** 曲线呈指数下降趋势，前 2 个 *epoch* 下降最快，之后逐渐平缓，最终接近 0.3。
- 2) **中图 (*Training Accuracy*):** 曲线从 0.50 快速上升，第 6 个 *epoch* 后超过 0.80，最后稳定在 0.90 以上。
- 3) **右图 (*Validation Accuracy*):** 曲线基本上升，第 4 个 *epoch* 后达到 0.60，之后在 0.60~0.75 之间波动

2. 预测结果可视化示例



随机抽取验证集中的 4 张图片，模型预测结果如下

图像	真实标签	预测标签	是否正确
图 1	<i>cow</i>	<i>Sheep</i>	X
图 2	<i>cow</i>	<i>cow</i>	✓
图 3	<i>cow</i>	<i>sheep</i>	X
图 4	<i>cow</i>	<i>sheep</i>	X

- 1) 分析：图示准确率为25%，这是由于图像数据集采集质量差，且样本数量过少造成的过拟合造成。该现象可通过重新采集或补充数据集或进行数据增强，增加`drop`层来缓解过拟合，增强模型泛化能力。

六、实验的收获及心得体会（或遇到的问题，如何解决）

1. 掌握了图像分类的完整流程：

从原始图像文件夹到可训练的数据加载器，再到 *CNN* 模型的设计、训练、验证和可视化，形成了一套标准化的代码模板，对后续其他图像分类任务有直接参考价值。

2. 理解了数据预处理的重要性：

- 1) *Resize* 统一图像尺寸，保证网络输入一致性。

- 2) *ToTensor* 将 *PIL* 图像转换为张量并自动归一化到 $[0,1]$ 。
- 3) 本实验未使用数据增强（如随机翻转、旋转），但可以通过 *transforms.Compose* 添加，进一步提升泛化能力。

七、实验代码

代码

```
import os
import json
import random
import sys
from tqdm import tqdm
import torch
from torch import nn, optim
from torch.utils.data import Dataset, DataLoader
from PIL import Image
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams['font.family'] = 'sans-serif'
plt.rcParams['axes.unicode_minus'] = False

# ----- 1. 读取与分割数据 -----
def read-split-data(root: str, val_rate: float = 0.2):
    random.seed(0)

    assert os.path.exists(root), f"dataset root: {root} does not exist."
    all_class = [cla for cla in os.listdir(root) if os.path.isdir(os.path.join(root, cla))]
    all_class.sort()
    class_indices = {k: v for v, k in enumerate(all_class)}
    with open('class-indices.json', 'w') as f:
        json.dump({v: k for k, v in class_indices.items()}, f, indent=4)

    train-images-path, train-images-label = [], []
    val-images-path, val-images-label = [], []
```

```

supported = ['.jpg', '.JPG', '.png', '.PNG', '.jpeg']

for cla in all_class:
    cla_path = os.path.join(root, cla)
    images = [os.path.join(root, cla, i) for i in
os.listdir(cla_path)
                if os.path.splitext(i)[-1] in supported]
    random.shuffle(images)
    val_num = int(len(images) * val_rate)
    val_path = images[:val_num]
    train_path = images[val_num:]
    train_images_path.extend(train_path)
    train_images_label.extend([class_indices[cla]] *
len(train_path))
    val_images_path.extend(val_path)
    val_images_label.extend([class_indices[cla]] * len(val_path))

    print(f"Total                images:
{len(train_images_path)+len(val_images_path)} ")
    print(f"Train samples: {len(train_images_path)}, Val samples:
{len(val_images_path)} ")
    return train_images_path, train_images_label, val_images_path,
val_images_label

# ----- 2. 自定义 Dataset -----
class MyDataSet(Dataset):
    def __init__(self, images_path: list, images_class: list,
transform=None):
        self.images_path = images_path
        self.images_class = images_class
        self.transform = transform

    def __len__(self):
        return len(self.images_path)

    def __getitem__(self, idx):
        img = Image.open(self.images_path[idx]).convert('RGB')
        label = self.images_class[idx]

```

```

        if self.transform:
            img = self.transform(img)
        return img, label

    @staticmethod
    def collate_fn(batch):
        images, labels = tuple(zip(*batch))
        images = torch.stack(images, dim=0)
        labels = torch.as_tensor(labels)
        return images, labels

# ----- 3. 卷积神经网络模型 -----
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(32 * 32 * 32, 128)
        self.fc2 = nn.Linear(128, 2)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = self.pool(nn.functional.relu(self.conv1(x)))
        x = self.pool(nn.functional.relu(self.conv2(x)))
        x = x.view(-1, 32 * 32 * 32)
        x = nn.functional.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x

# ----- 4. 训练函数 -----
def trainer(model, optimizer, data_loader, epoch, device):
    model.train()
    total_loss, total_acc = 0.0, 0.0
    criterion = nn.CrossEntropyLoss()
    pbar = tqdm(data_loader, file=sys.stdout)
    for images, labels in pbar:

```

```

        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        preds = torch.argmax(outputs, dim=1)
        acc = (preds == labels).sum().item() / labels.size(0)
        total_loss += loss.item()
        total_acc += acc
        pbar.set_description(f"[Epoch {epoch}] Loss: {loss.item():.3f} Acc: {acc:.3f}")
    return total_loss/len(data_loader), total_acc/len(data_loader)

# ----- 5. 验证函数 -----
def evaluator(model, data_loader, device):
    model.eval()
    total_correct = 0
    with torch.no_grad():
        for images, labels in data_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            preds = torch.argmax(outputs, dim=1)
            total_correct += (preds == labels).sum().item()
    return total_correct / len(data_loader.dataset)

# ----- 6. 预测可视化 -----
def visualize_predictions(model, data_loader, device, class_names,
num_samples=4):
    model.eval()
    images, labels = next(iter(data_loader))
    images, labels = images.to(device), labels.to(device)
    with torch.no_grad():
        outputs = model(images)
        preds = torch.argmax(outputs, dim=1)
    plt.figure(figsize=(12, 8))
    for i in range(min(num_samples, len(images))):
        plt.subplot(2, 2, i+1)

```

```

        img = images[i].cpu().permute(1, 2, 0).numpy()
        img = img * 255
        img = img.astype(np.uint8)
        plt.imshow(img)
        plt.title(f"True:      {class_names[labels[i].item()]},      Pred:
{class_names[preds[i].item()]}")
        plt.axis('off')
    plt.tight_layout()
    plt.show()

# ----- 7. 主程序 -----
if __name__ == "__main__":
    data_root = "./data"
    batch_size = 16
    epochs = 10
    lr = 0.001
    img_size = 128

    # 加载数据
    train_paths,      train_labels,      val_paths,      val_labels      =
read_split_data(data_root)

    # 加载类别名称
    with open('class_indices.json', 'r') as f:
        class_indices = json.load(f)
    class_names      =      [class_indices[str(i)]      for      i      in
range(len(class_indices))]

    # 数据预处理
    transform = transforms.Compose([
        transforms.Resize((img_size, img_size)),
        transforms.ToTensor()
    ])

    # 创建 DataLoader
    train_dataset = MyDataSet(train_paths, train_labels, transform)
    val_dataset = MyDataSet(val_paths, val_labels, transform)
    train_loader = DataLoader(train_dataset, batch_size=batch_size,

```

```

shuffle=True,

                                collate_fn=train_dataset.collate_fn)
val_loader = DataLoader(val_dataset, batch_size=batch_size,
                        collate_fn=val_dataset.collate_fn)

# 模型、优化器、设备
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {device}")
model = Net().to(device)
optimizer = optim.Adam(model.parameters(), lr=lr)

# 训练循环
train_losses, train_accs, val_accs = [], [], []
for epoch in range(1, epochs+1):
    train_loss, train_acc = trainer(model, optimizer,
train_loader, epoch, device)
    val_acc = evaluator(model, val_loader, device)
    train_losses.append(train_loss)
    train_accs.append(train_acc)
    val_accs.append(val_acc)
    print(f"Epoch {epoch} / {epochs} | Val Acc: {val_acc:.4f}")

# 保存模型
torch.save(model.state_dict(), "animal_classifier.pth")
print("模型已保存为 animal_classifier.pth")

# 绘制训练曲线
plt.figure(figsize=(15, 4))
plt.subplot(131)
plt.plot(range(1, epochs+1), train_losses, label='Train Loss')
plt.xlabel('Epoch'); plt.ylabel('Loss'); plt.title('Training
Loss'); plt.legend()
plt.subplot(132)
plt.plot(range(1, epochs+1), train_accs, label='Train ACC')
plt.xlabel('Epoch'); plt.ylabel('Accuracy'); plt.title('Training
Accuracy'); plt.legend()
plt.subplot(133)
plt.plot(range(1, epochs+1), val_accs, label='Val Acc')

```

```
plt.xlabel('Epoch'); plt.ylabel('Accuracy');  
plt.title('Validation Accuracy'); plt.legend()  
plt.tight_layout()  
plt.show()  
  
# 可视化预测结果  
visualize_predictions(model, val_loader, device, class_names)
```